

DAY 7

Environment Variables, Parameter Expansion & Tilde Expansion – Detailed Notes

This topic builds directly on shell variables. The key idea is that the shell can **store information, modify it, extract parts of it, and use special shortcuts to refer to directories.**

1. Environment Variables

An **environment variable** is a variable that is available to the shell and, when exported, can also be inherited by processes launched from that shell.

Common environment-related variables include:

```
PATH
HOME
USER
SHELL
PWD
OLDPWD
PS1
```

Two particularly important ones are:

- **PATH** → controls where the shell searches for commands.
 - **PS1** → controls the appearance of the interactive shell prompt.
-

2. **PATH** Environment Variable

PATH is one of the **most important environment variables in Linux.**

It contains a list of directories where the shell looks for executable commands.

Check it with:

```
echo "${PATH}"
```

You may see something like:

```
/usr/local/sbin:/usr/local/bin:/usr/bin:/usr/sbin:/sbin
```

The directories are separated by a colon (:).

Conceptually:

```
PATH
|
├─ /usr/local/sbin
├─ /usr/local/bin
├─ /usr/bin
├─ /usr/sbin
└─ /sbin
```

2.1 How PATH Works

Suppose you type:

```
ls
```

You don't normally type:

```
/usr/bin/ls
```

So how does the shell find `ls`?

The shell searches the directories listed in `PATH`.

For example:

```
User types:
  ls
  ↓
Shell searches PATH
  ↓
/usr/local/bin
```

```
↓  
/usr/bin  
↓  
finds /usr/bin/ls  
↓  
executes ls
```

You can find the location of a command using:

```
which ls
```

or:

```
command -v ls
```

Example:

```
/usr/bin/ls
```

3. Why `PATH` Matters in Security

`PATH` is particularly important in cybersecurity.

Suppose a script executes:

```
backup
```

instead of:

```
/usr/bin/backup
```

The shell searches `PATH` to find the executable.

If an attacker can manipulate the `PATH` and place a malicious executable earlier in the search order, the wrong program could potentially be executed.

For example:

```
PATH=/tmp:/usr/bin:/bin
```

If `/tmp` contains a malicious executable called:

```
ls
```

the shell may find:

```
/tmp/ls
```

before:

```
/usr/bin/ls
```

This is one reason **PATH manipulation/hijacking** is an important security concept.

SOC takeaway: Unexpected changes to `PATH` can be a useful indicator during investigation, especially for suspicious scripts, persistence, or privilege-related activity.

4. `PS1` Environment Variable

`PS1` controls the **primary interactive shell prompt**.

Check it:

```
echo "${PS1}"
```

A Bash prompt might conceptually contain:

```
\u@\h:\w\$_
```

These are prompt escape sequences.

For example:

Sequence	Meaning
\u	Username
\h	Hostname
\w	Current working directory
\\$	\$ for normal user, # for root

So a prompt could appear as:

```
sourya@cyber30:~/Documents$
```

The prompt is being generated according to the value of `PS1`.

5. Changing `PS1`

You can temporarily change the prompt:

```
PS1="MyShell> "
```

The prompt becomes:

```
MyShell>
```

Another example:

```
PS1="\u@\h:\w\$ "
```

This creates a prompt containing:

```
username@hostname:directory$
```

Important

Changing `PS1` normally affects the **current shell session** unless you put the configuration into a shell startup file such as:

```
~/ .bashrc
```

6. Parameter Expansion

Parameter expansion is the process of obtaining and manipulating the value stored in a shell parameter.

Basic expansion:

```
name="Sourya"  
echo "${name}"
```

Output:

```
Sourya
```

But Bash can do much more than simply retrieve the value.

You can use parameter expansion to:

- Convert text to lowercase
- Convert text to uppercase
- Find string length
- Extract portions of strings
- Replace/remove patterns
- Provide default values
- And more

7. Lowercase Conversion

Bash parameter expansion can be used to convert text to lowercase.

Suppose:

```
name="SOURYA DUTTA"
```

You can use:

```
echo "${name,,}"
```

Output:

```
sourya dutta
```

Important Syntax

```
${variable,,}
```

means:

| Convert the value of the variable to lowercase.

Example:

```
text="HELLO WORLD"  
  
echo "${text,,}"
```

Output:

```
hello world
```

8. Uppercase Conversion

Similarly, use:

```
${variable^^}
```

to convert the value to uppercase.

Example:

```
name="sourya dutta"  
  
echo "${name^^}"
```

Output:

```
SOURYA DUTTA
```

Remember

```
${var,,}    → lowercase  
${var^^}    → uppercase
```

This is a very useful Bash feature because you don't need an external command such as `tr` for simple case conversion.

9. String Length

You can find the number of characters in a variable using:

```
${#variable}
```

Example:

```
name="Sourya"  
  
echo "${#name}"
```

Output:

```
6
```

Because:

```
S o u r y a  
1 2 3 4 5 6
```

9.1 Another Example


```
message="Hello World"
```

```
echo "${#message}"
```

The result is:

```
11
```

Why?

```
Hello → 5
```

```
space → 1
```

```
World → 5
```

```
Total → 11
```

Important Syntax

```
${#variable}
```

means:

| Return the length of the variable's value.

10. String Slicing

String slicing means extracting a portion of a string.

The general syntax is:

```
${variable:start:length}
```

Where:

- `variable` → variable containing the string
 - `start` → starting position
 - `length` → number of characters to extract
-

11. Basic String Slicing Example

Consider:

```
name="SouryaDutta"
```

Character positions are:

```
S o u r y a D u t t a  
0 1 2 3 4 5 6 7 8 9 10
```

Now:

```
echo "${name:0:6}"
```

Output:

```
Sourya
```

Why?

```
Start = 0  
Length = 6
```

So Bash extracts:

```
S o u r y a
```

12. Another Slicing Example

```
name="SouryaDutta"
```

```
echo "${name:6:5}"
```

Output:

```
Dutta
```

Because:

```
S o u r y a D u t t a
0 1 2 3 4 5 6 7 8 9 10
          ↑
        start
```

Starting at position 6, extract 5 characters.

13. Slicing Without Specifying Length

You can also specify only the starting position:

```
${variable:start}
```

Example:

```
name="SouryaDutta"

echo "${name:6}"
```

Output:

```
Dutta
```

This means:

| Start at position 6 and continue to the end.

14. Negative Indexing

Bash also supports negative offsets for strings.

For example:

```
name="SouryaDutta"

echo "${name: -5}"
```

Output:

```
Dutta
```

Notice the **space** before `-5`.

This tells Bash that the negative number is an offset from the end of the string.

Conceptually:

```
S o u r y a D u t t a
-11                -1
```

This can be useful when you want to extract the last few characters.

15. Parameter Expansion Quick Reference

These are worth memorizing:

Syntax	Meaning
<code>\${var}</code>	Get value
<code>\${#var}</code>	Get length
<code>\${var,,}</code>	Convert to lowercase
<code>\${var^^}</code>	Convert to uppercase
<code>\${var:start}</code>	Slice from <code>start</code> to end
<code>\${var:start:length}</code>	Extract <code>length</code> characters

Example:

```
name="SouryaDutta"

echo "${name}"
echo "${name,,}"
echo "${name^^}"
echo "${#name}"
echo "${name:0:6}"
```

16. Tilde (~) Expansion

The `~` symbol is called **tilde**.

In Bash, tilde expansion provides a convenient shortcut for referring to home directories.

The most common form is:

```
~
```

which normally expands to the current user's home directory.

For example, if:

```
HOME=/home/sourya
```

then:

```
cd ~
```

is effectively equivalent to:

```
cd /home/sourya
```

17. `~` and `$HOME`

These are closely related:

```
~
```

and:

```
${HOME}
```

For example:

```
echo ~
```

might produce:

```
/home/sourya
```

while:

```
echo "${HOME}"
```

also produces:

```
/home/sourya
```

So:

```
~ → current user's home directory
```

18. Why `~` Is Useful

Instead of writing:

```
cd /home/sourya/Documents
```

you can write:

```
cd ~/Documents
```

This is shorter and more portable.

For example:

```
ls ~/Downloads
```

means:

| List the `Downloads` directory inside my home directory.

19. Tilde Expansion for Other Users

You can also use:

```
~username
```

to refer to another user's home directory, provided that user exists and the system can resolve their home directory.

For example:

```
~john
```

could expand to:

```
/home/john
```

if `john` is a valid user with that home directory.

Example:

```
cd ~john
```

means:

| Go to John's home directory.

20. Tilde Can Verify User Existence

This is an interesting practical use of tilde expansion.

Suppose:

```
~john
```

is expanded successfully.

That indicates that the system can resolve `john` as a user/home-directory entry.

If the user doesn't exist or cannot be resolved, the expansion may not work as expected.

For example:

```
echo ~john
```

could produce:

```
/home/john
```

if the account exists.

This makes tilde expansion useful for understanding how the shell resolves users and their home directories.

Important: Tilde expansion itself is not a security mechanism for validating users. It is simply shell expansion based on user/account information.

21. `~+` – Shortcut for `$PWD`

Bash provides another tilde expansion:

```
~+
```

It represents the current working directory.

It is essentially associated with:

```
${PWD}
```

Example:

```
echo ~+
```

might output:


```
/home/sourya/Documents
```

while:

```
echo "${PWD}"
```

might give:

```
/home/sourya/Documents
```

So remember:

```
~+ → PWD
```

22. `~-` – Shortcut for `$OLDPWD`

Another useful form is:

```
~-
```

This refers to the previous working directory stored in:

```
OLDPWD
```

So conceptually:

```
~- → OLDPWD
```

Example:

```
echo "${OLDPWD}"
```

might show:

```
/home/sourya
```

and:

```
echo ~-
```

can refer to that previous directory.

23. OLDPWD

OLDPWD stores the **previous working directory**.

Suppose you start here:

```
/home/sourya
```

Then execute:

```
cd /tmp
```

Now:

```
pwd
```

shows:

```
/tmp
```

and:

```
echo "${OLDPWD}"
```

may show:

```
/home/sourya
```

This is also why the following command is so useful:

```
cd -
```

It switches to the previous working directory.

24. `PWD`, `OLDPWD`, and Tilde Expansion

These concepts are connected:

```
PWD
↓
Current working directory

OLDPWD
↓
Previous working directory
```

And Bash provides:

```
~+
↓
PWD

~-
↓
OLDPWD
```

So:

```
echo "${PWD}"
echo ~+
```

refer to the current directory.

And:

```
echo "${OLDPWD}"
echo ~-
```

refer to the previous directory.

25. Complete Example

Consider:

```
#!/bin/bash

name="Sourya Dutta"

echo "Name: ${name}"
echo "Lowercase: ${name,,}"
echo "Uppercase: ${name^^}"
echo "Length: ${#name}"

echo "First 6 characters: ${name:0:6}"

echo "Home: ${HOME}"
echo "Home using tilde: ~"

echo "Current directory: ${PWD}"
echo "Current directory using ~+: ~+"
echo "Previous directory: ${OLDPWD}"
echo "Previous directory using ~-: ~-"
```

This demonstrates several different Bash concepts:

Variables
↓
Parameter Expansion
↓

Value
Lowercase
Uppercase
Length
Slicing

↓
Tilde Expansion
↓
Home / PWD / OLDPWD

26. Important Difference: Expansion Happens Before Execution

One of the most important concepts to understand is that the shell performs expansions while processing the command before the command is executed.

For example:

```
name="Sourya"  
echo "${name}"
```

The shell expands:

```
"${name}"
```

into:

```
Sourya
```

and then `echo` receives the resulting argument.

Similarly:

```
cd ~/Documents
```

uses tilde expansion before `cd` operates on the resulting path.
